

PRACTICAL 7

Aim

To implement performance optimization techniques using Caching and Partitioning in Apache Spark with PySpark.

The practical focuses on loading the MovieLens Ratings dataset into a Spark DataFrame, examining its schema and data, calculating average movie ratings, measuring execution time before and after caching, applying `cache()` and `persist(StorageLevel.DISK_ONLY)`, comparing different partition counts, analyzing `repartition()` and `coalesce()`, and observing the effect of these techniques on Spark performance.

Problem Definition

A movie recommendation company processes millions of user ratings to generate personalized movie recommendations. The recommendation system repeatedly processes movie rating data to calculate statistics such as average movie ratings and rating counts.

As the volume of user interaction data increases, repeated processing of the same dataset results in increased execution time and computational overhead. Therefore, Apache Spark caching and partitioning techniques are used to improve application performance.

The task is to load the MovieLens Ratings dataset into a Spark DataFrame and perform performance optimization using caching and partitioning. The practical includes creating a SparkSession, displaying the dataset and schema, calculating average movie ratings, measuring execution time before caching, applying `cache()`, triggering caching using `count()`, measuring execution time after caching, comparing `cache()` with `persist(StorageLevel.DISK_ONLY)`, repartitioning the dataset into 2, 4 and 8 partitions, comparing `coalesce()` and `repartition()`, and observing scalability using a larger ratings dataset.

Objectives

- Understand Apache Spark performance optimization.
- Create and configure a SparkSession.
- Load the MovieLens Ratings dataset into a Spark DataFrame.
- Display and examine the dataset.
- Display the schema of the DataFrame.
- Check the initial number of partitions.
- Calculate average movie ratings.
- Measure execution time before caching.
- Apply `cache()` to the DataFrame.
- Trigger caching using `count()`.
- Measure execution time after caching.
- Compare performance before and after caching.
- Use `persist(StorageLevel.DISK_ONLY)`.

- Compare cache() and DISK_ONLY persistence.
- Repartition the dataset into 2, 4 and 8 partitions.
- Compare execution times for different partition counts.
- Compare coalesce() and repartition().
- Create a larger ratings dataset.
- Observe scalability and performance.
- Understand Spark lazy evaluation.
- Analyze how caching and partitioning affect Spark performance.

Output

The first screenshot shows the Jupyter Notebook interface with the following code and output:

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count
from pyspark.storagelevel import StorageLevel
import time

[2]: spark = SparkSession.builder \
    .appName("Practical7_PerformanceOptimization") \
    .master("local[*]") \
    .getOrCreate()

spark

[2]: SparkSession - in memory

SparkContext

Spark UI

Version          v4.2.0
Master           local[*]
AppName          Practical7_PerformanceOptimization

[3]: ratings_path = "ratings.csv"

ratings_df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(ratings_path)

ratings_df.show(10)
```

The output shows the first 10 rows of the ratings dataset:

userId	movieId	rating	_c3
1	1	4.0	30-07-2000 18:45:03
1	3	4.0	30-07-2000 18:45:03
1	6	4.0	30-07-2000 18:45:03
1	47	5.0	30-07-2000 18:45:03
1	50	5.0	30-07-2000 18:45:03
1	70	5.0	30-07-2000 18:45:03
1	101	5.0	30-07-2000 18:45:03
1	110	4.0	30-07-2000 18:45:03
1	151	5.0	30-07-2000 19:07:11
1	157	5.0	30-07-2000 19:08:20

only showing top 10 rows

The second screenshot shows the following code and output:

```
[4]: ratings_df.printSchema()

root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)
 |-- _c3: string (nullable = true)

[5]: print("Number of records:", ratings_df.count())

Number of records: 100036

[6]: ratings_df.describe().show()

summary|      userId|      movieId|      rating|      _c3|
-----|-----|-----|-----|-----|
| count|      100036|      100036|      100036|      100036|
| mean|326.1275635856676|19435.2957177992| 3.581556983616962|      NULL|
| stddev|182.6184914431904|35536.9871987983|1.8425292390886342|      NULL|
| min|      1|      1|      0.5|01-01-1997 18:10:51|
| max|      610|      193689|      5.0|31-12-2016 21:57:02|

[7]: initial_partitions = ratings_df.rdd.getNumPartitions()

print("Initial number of partitions:", initial_partitions)

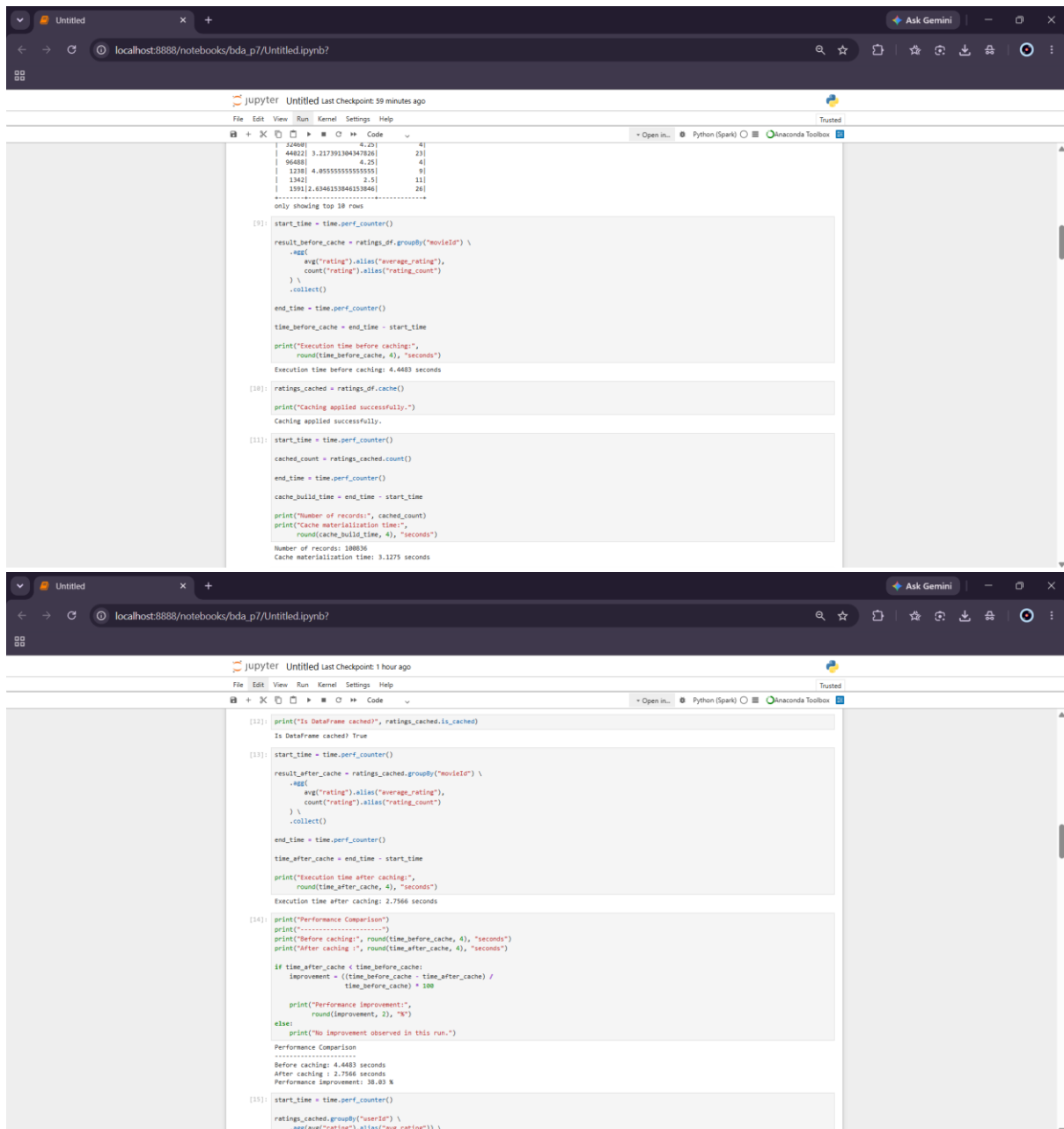
Initial number of partitions: 1

[8]: average_ratings = ratings_df.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    )

average_ratings.show(10)
```

The output shows the schema, record count, describe output, and the creation of a new dataset with grouped average ratings:

movieId	average_rating	rating_count
1	3.487878787878788	169
23661	3.464	25
31751	3.58	75
18881	3.369047619047619	42
12460	4.15	4
46801	4.117001041700104	91



The image displays two screenshots of a Jupyter Notebook interface, showing the execution of a Spark query and the performance comparison of the query with and without caching.

Top Screenshot: The notebook shows the execution of a Spark query. The output displays the top 5 rows of the result, followed by a message indicating that only the top 50 rows are shown. The code includes a performance measurement section (lines 19-27) and a caching operation (lines 28-30).

```
[19]: start_time = time.perf_counter()

result_before_cache = ratings_df.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()
time_before_cache = end_time - start_time

print("Execution time before caching:",
      round(time_before_cache, 4), "seconds")
Execution time before caching: 4.4683 seconds

[20]: ratings_cached = ratings_df.cache()

print("Caching applied successfully.")
Caching applied successfully.

[21]: start_time = time.perf_counter()

cached_count = ratings_cached.count()
end_time = time.perf_counter()
cache_build_time = end_time - start_time

print("Number of records:", cached_count)
print("Cache materialization time:",
      round(cache_build_time, 4), "seconds")
Number of records: 100036
Cache materialization time: 3.1279 seconds
```

Bottom Screenshot: The notebook shows the execution of the same Spark query, but with caching. The code includes a performance measurement section (lines 12-14) and a caching operation (lines 15-17). The output displays the execution time after caching, followed by a performance comparison section (lines 18-24) and the execution of the query (lines 25-27).

```
[12]: print("Is Dataframe cached?", ratings_cached.is_cached)
Is Dataframe cached? True

[13]: start_time = time.perf_counter()

result_after_cache = ratings_cached.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()
time_after_cache = end_time - start_time

print("Execution time after caching:",
      round(time_after_cache, 4), "seconds")
Execution time after caching: 2.7566 seconds

[14]: print("Performance Comparison")
print("-----")
print("Before caching:", round(time_before_cache, 4), "seconds")
print("After caching :", round(time_after_cache, 4), "seconds")

if time_after_cache < time_before_cache:
    improvement = ((time_before_cache - time_after_cache) /
                  time_before_cache) * 100

    print("Performance Improvement:",
          round(improvement, 2), "%")
else:
    print("No Improvement observed in this run.")

Performance Comparison
-----
Before caching: 4.4683 seconds
After caching : 2.7566 seconds
Performance Improvement: 38.83 %

[15]: start_time = time.perf_counter()

ratings_cached.groupBy("userId") \
    .agg(avg("rating").alias("avg_rating")) \
```

The image displays two screenshots of a Jupyter Notebook running on a local host (localhost:8888). The notebook is titled "Untitled" and is using the Python (Spark) kernel. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations, running code, and viewing output.

Top Screenshot: The code cell contains the following Python code:

```
ratings_cached.groupBy("userId") \
    .agg(avg("rating").alias("avg_rating")) \
    .count()

end_time = time.perf_counter()

print("First operation time:",
      round(end_time - start_time, 4), "seconds")

First operation time: 2.5212 seconds

[16]: start_time = time.perf_counter()

ratings_cached.groupBy("userId") \
    .agg(avg("rating").alias("avg_rating")) \
    .count()

end_time = time.perf_counter()

print("Repeated operation time:",
      round(end_time - start_time, 4), "seconds")

Repeated operation time: 2.0319 seconds

[17]: ratings_cached.storageLevel

[17]: StorageLevel(True, True, False, True, 1)

[18]: print(ratings_cached.storageLevel)

Disk Memory Deserialized in 1 Replicated

[19]: ratings_cached.unpersist()

print("Cache removed.")

Cache removed.

[20]: ratings_disk = ratings_df.persist(StorageLevel.DISK_ONLY)

print("Storage level:", ratings_disk.storageLevel)

Storage level: Disk Serialized in 1 Replicated

[21]: start_time = time.perf_counter()
```

Bottom Screenshot: The code cell contains the following Python code:

```
[21]: start_time = time.perf_counter()

ratings_disk.count()

end_time = time.perf_counter()

print("DISK_ONLY materialization time:",
      round(end_time - start_time, 4), "seconds")

DISK_ONLY materialization time: 2.363 seconds

[22]: start_time = time.perf_counter()

ratings_disk.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()

disk_time = end_time - start_time

print("Execution time using DISK_ONLY:",
      round(disk_time, 4), "seconds")

Execution time using DISK_ONLY: 2.3312 seconds

[23]: ratings_disk.unpersist()

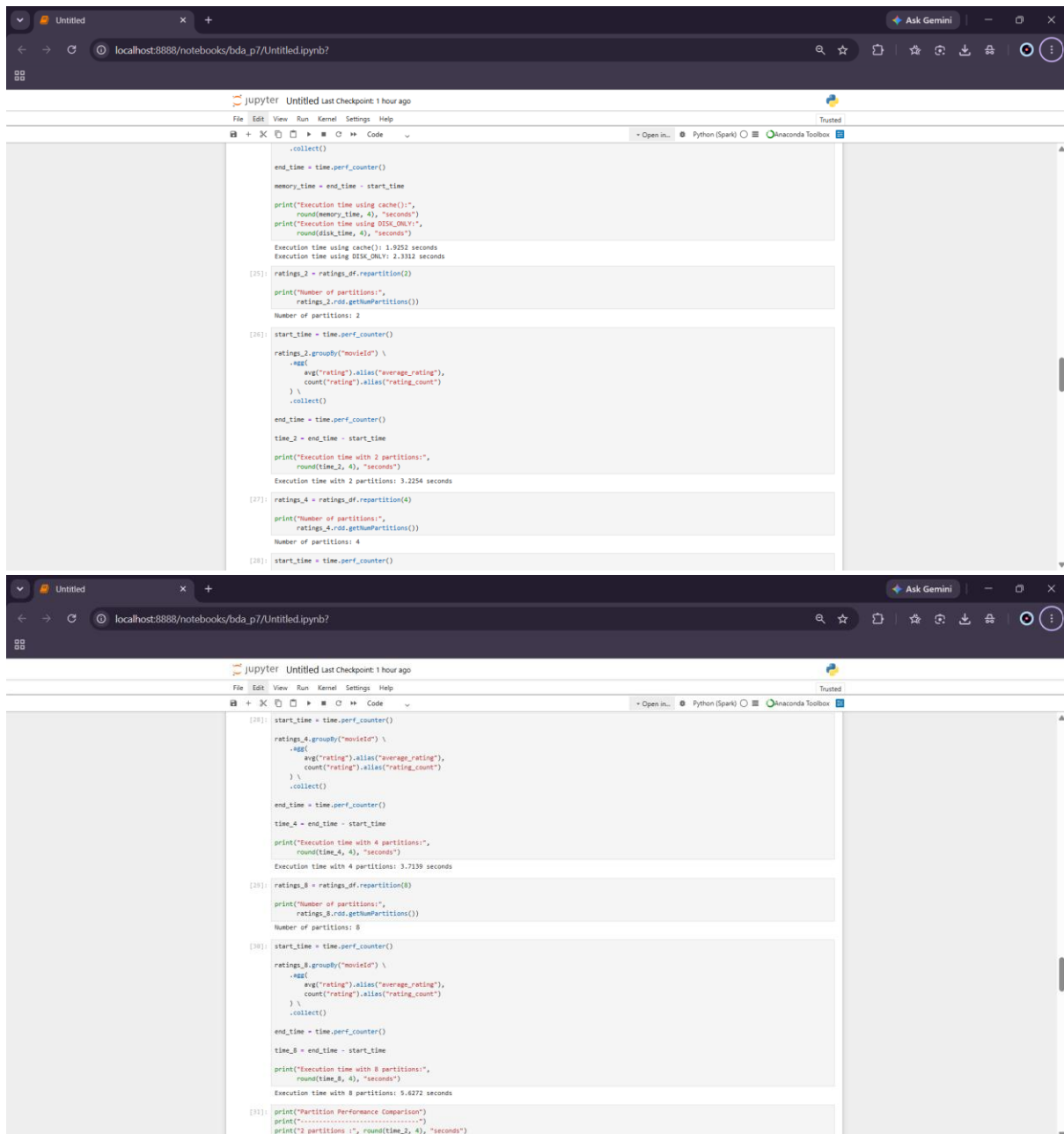
ratings_memory = ratings_df.cache()

# Materialize cache
ratings_memory.count()

[23]: 1000036

[24]: start_time = time.perf_counter()

ratings_memory.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()
```



```
.collect()

end_time = time.perf_counter()
memory_time = end_time - start_time
print("Execution time using cache():",
      round(memory_time, 4), "seconds")
print("Execution time using DISK_ONLY:",
      round(disk_time, 4), "seconds")
Execution time using cache(): 1.9252 seconds
Execution time using DISK_ONLY: 2.3312 seconds

[25]: ratings_2 = ratings_df.repartition(2)

print("Number of partitions:",
      ratings_2.rdd.getNumPartitions())
Number of partitions: 2

[26]: start_time = time.perf_counter()

ratings_2.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()
time_2 = end_time - start_time
print("Execution time with 2 partitions:",
      round(time_2, 4), "seconds")
Execution time with 2 partitions: 3.2254 seconds

[27]: ratings_4 = ratings_df.repartition(4)

print("Number of partitions:",
      ratings_4.rdd.getNumPartitions())
Number of partitions: 4

[28]: start_time = time.perf_counter()

ratings_4.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()
time_4 = end_time - start_time
print("Execution time with 4 partitions:",
      round(time_4, 4), "seconds")
Execution time with 4 partitions: 3.7139 seconds

[29]: ratings_8 = ratings_df.repartition(8)

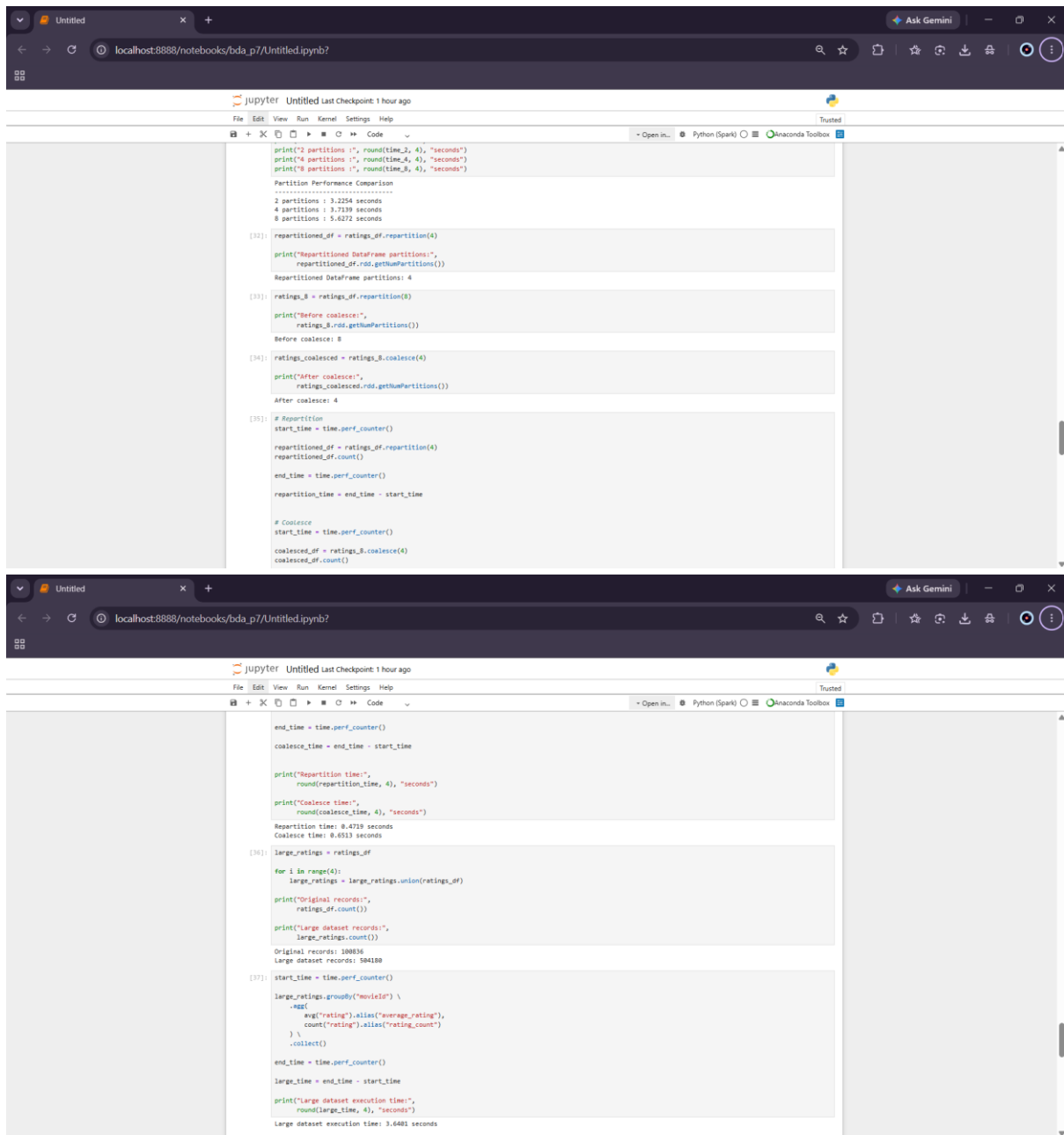
print("Number of partitions:",
      ratings_8.rdd.getNumPartitions())
Number of partitions: 8

[30]: start_time = time.perf_counter()

ratings_8.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()
time_8 = end_time - start_time
print("Execution time with 8 partitions:",
      round(time_8, 4), "seconds")
Execution time with 8 partitions: 5.6272 seconds

[31]: print("Partition Performance Comparison")
print("-----")
print("2 partitions :", round(time_2, 4), "seconds")
print("4 partitions :", round(time_4, 4), "seconds")
print("8 partitions :", round(time_8, 4), "seconds")
```



The image displays two screenshots of a Jupyter Notebook interface, likely running on a local host (localhost:8888). The notebook is titled "Untitled" and shows code for working with Spark partitions and coalescing.

Top Screenshot:

```
print("2 partitions :", round(time_2, 4), "seconds")
print("4 partitions :", round(time_4, 4), "seconds")
print("8 partitions :", round(time_8, 4), "seconds")

Partition Performance Comparison
.....
2 partitions : 3.2254 seconds
4 partitions : 3.7139 seconds
8 partitions : 5.6272 seconds

[32]: repartitioned_df = ratings_df.repartition(4)

print("Repartitioned DataFrame partitions:",
      repartitioned_df.rdd.getNumPartitions())
Repartitioned DataFrame partitions: 4

[33]: ratings_8 = ratings_df.repartition(8)

print("Before coalesce:",
      ratings_8.rdd.getNumPartitions())
Before coalesce: 8

[34]: ratings_coalesced = ratings_8.coalesce(4)

print("After coalesce:",
      ratings_coalesced.rdd.getNumPartitions())
After coalesce: 4

[35]: # Repartition
start_time = time.perf_counter()
repartitioned_df = ratings_df.repartition(4)
repartitioned_df.count()

end_time = time.perf_counter()

repartition_time = end_time - start_time

# Coalesce
start_time = time.perf_counter()
coalesced_df = ratings_8.coalesce(4)
coalesced_df.count()
```

Bottom Screenshot:

```
end_time = time.perf_counter()
coalesce_time = end_time - start_time

print("Repartition time:",
      round(repartition_time, 4), "seconds")

print("Coalesce time:",
      round(coalesce_time, 4), "seconds")
Repartition time: 0.4719 seconds
Coalesce time: 0.6513 seconds

[36]: large_ratings = ratings_df

for i in range(4):
    large_ratings = large_ratings.union(ratings_df)

print("Original records:",
      ratings_df.count())

print("Large dataset records:",
      large_ratings.count())
Original records: 100036
Large dataset records: 504180

[37]: start_time = time.perf_counter()

large_ratings.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()

end_time = time.perf_counter()

large_time = end_time - start_time

print("Large dataset execution time:",
      round(large_time, 4), "seconds")
Large dataset execution time: 3.6401 seconds
```

```

[38]: large_cached = large_ratings.cache()
start_time = time.perf_counter()
large_cached.count()
end_time = time.perf_counter()
print("Cache materialization time:",
      round(end_time - start_time, 4), "seconds")
Cache materialization time: 1.688 seconds

[39]: start_time = time.perf_counter()
large_cached.groupBy("movieId") \
    .agg(
        avg("rating").alias("average_rating"),
        count("rating").alias("rating_count")
    ) \
    .collect()
end_time = time.perf_counter()
print("Large dataset cached execution time:",
      round(end_time - start_time, 4), "seconds")
Large dataset cached execution time: 3.5879 seconds

[40]: print("Large dataset partitions:",
        large_ratings.rdd.getNumPartitions())
Large dataset partitions: 5

[41]: large_R = large_ratings.repartition(8)
print("After repartitioning:",
      large_R.rdd.getNumPartitions())
After repartitioning: 8

[42]: ratings_df.groupBy("movieId") \
    .agg(avg("rating").alias("average_rating")) \
    .collect(True)
== Parsed Logical Plan ==
'Aggregate ['movieId', 'movieId', 'avg(rating) AS average_rating#4552]

== Analyzed Logical Plan ==
Aggregate [movieId: int, average_rating: double]
Aggregate [movieId#18], [movieId#18, avg(rating#19) AS average_rating#4552]
Relation [user#17,movieId#18,rating#19,_c3#20] csv

== Optimized Logical Plan ==
Aggregate [movieId#18], [movieId#18, avg(rating#19) AS average_rating#4552]
Project [movieId#18, ratings#19]
-- InMemoryRelation [user#17, movieId#18, rating#19, _c3#20], StorageLevel(disk, memory, deserialized, 1 replicas)
-- FileScan csv [user#17,movieId#18,rating#19,_c3#20] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex[1 paths]#f1
e/C:/Users/shiva/bda_p7/ratings.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<userId:int,movieId:int,rating:double,_c3:string>

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
-- HashAggregate(keys=[movieId#18], functions=[avg(rating#19)], output=[movieId#18, average_rating#4552])
-- Exchange hashpartitioning(movieId#18, 200), EXCHANGE_ID(1000000001), plan_id=1710
-- HashAggregate(keys=[movieId#18], functions=[partial_avg(rating#19)], output=[movieId#18, sum#4628, count#4621])
-- InMemoryTableScan [movieId#18, rating#19]
-- InMemoryRelation [user#17, movieId#18, rating#19, _c3#20], StorageLevel(disk, memory, deserialized, 1 replicas)
-- FileScan csv [user#17,movieId#18,rating#19,_c3#20] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex
[1 paths]#f1e/C:/Users/shiva/bda_p7/ratings.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<userId:int,movieId:int,rating:double,_c3:string>

[43]: large_cached.unpersist()
print("Cache cleared.")
Cache cleared.

[44]: ratings_memory.unpersist()
print("All cached DataFrames removed.")
All cached DataFrames removed.

[45]: spark.stop()

```

Key Questions / Analysis

Q1. What is caching in Apache Spark and why does it improve performance?

Caching is a performance optimization technique in Apache Spark that stores a computed DataFrame or RDD so that it can be reused for subsequent operations.

When the same dataset is used repeatedly, Spark can access the cached data instead of recomputing the dataset from the original source.

Caching can therefore:

1. Reduce repeated computation.

2. Reduce data processing time.
3. Improve performance for repeated operations.
4. Reduce the need to recompute transformations.
5. Improve interactive analytics and iterative processing.

In this practical, `cache()` is used to store the MovieLens ratings DataFrame for repeated analysis.

Q2. What is the difference between `cache()` and `persist()`?

`cache()`

The `cache()` method stores a DataFrame or RDD using Spark's default caching storage level.

Example:

```
ratings_cached = ratings_df.cache()
```

`persist()`

The `persist()` method allows the programmer to specify the storage level.

Example:

```
from pyspark.storagelevel import StorageLevel
ratings_disk = ratings_df.persist(StorageLevel.DISK_ONLY)
```

Therefore:

<code>cache()</code>	<code>persist()</code>
Uses Spark's default storage level	Allows storage level to be specified
Simple to use	Provides more control
Suitable for frequently reused data	Useful when a specific storage strategy is required

Q3. How does Spark partition data internally?

Spark divides a dataset into multiple partitions. Each partition represents a portion of the complete dataset.

Spark can process different partitions in parallel using different tasks and available CPU resources.

For example:

Dataset

```
|  
+---- Partition 1  
  
|  
+---- Partition 2  
  
|  
+---- Partition 3  
  
|  
+---- Partition 4
```

The number of partitions can affect the amount of parallelism and the execution performance of a Spark application.

Q4. Why does `repartition()` improve scalability?

The `repartition()` operation redistributes data into the specified number of partitions.

For example:

```
ratings_4 = ratings_df.repartition(4)
```

This creates four partitions.

With appropriate partitioning, Spark can distribute processing tasks across multiple CPU cores. This can improve parallel processing for larger datasets.

However, excessive partitioning can also introduce additional task scheduling and shuffle overhead.

Q5. How does lazy evaluation affect Spark performance?

Spark uses **lazy evaluation**, which means transformations are not executed immediately.

For example:

```
df2 = df.filter(df.rating > 3)
```

The `filter()` operation does not immediately process all records.

Spark waits until an action such as:

```
df2.count()
```

or:

```
df2.collect()
```

is executed.

Lazy evaluation allows Spark to construct and optimize an execution plan before actually performing the computation.

Supplementary Problems

1. Compare the performance of `cache()` and `persist(StorageLevel.DISK_ONLY)`.
2. Repartition the dataset into 2, 4 and 8 partitions and compare execution time.
3. Compare `coalesce()` and `repartition()`.
4. Create a larger ratings dataset and observe scalability.
5. Analyze Spark's execution plan using `explain()`.
6. Compare execution time before and after caching.

Conclusion

The Performance Optimization Using Caching and Partitioning practical was successfully completed using PySpark and the MovieLens Ratings dataset. The dataset was loaded into a Spark DataFrame and its schema and partitions were examined. Average movie ratings were calculated and execution time was measured before and after applying caching.

The `cache()` method was used to store frequently accessed data, while `persist(StorageLevel.DISK_ONLY)` was used to study disk-based persistence. Different partition counts, including 2, 4 and 8 partitions, were tested to analyze their effect on execution time. The practical also compared `coalesce()` and `repartition()` and examined the performance of a larger ratings dataset.

The experiment demonstrated the importance of caching, persistence, partitioning, lazy evaluation and parallel processing in optimizing Apache Spark applications. These techniques help improve the efficiency and scalability of data processing workloads, particularly when datasets are large or repeatedly accessed.

https://github.com/Bhumi3009/D25DIT090_BDA_SHP/tree/main